

Description Editorial Solutions Submissions

27. Remove Element

Easy Topics Companies Hint

Given an integer array `nums` and an integer `val`, remove all occurrences of `val` in `nums` **in-place**. The order of the elements may be changed. Then return *the number of elements in `nums` which are not equal to `val`*.

Consider the number of elements in `nums` which are not equal to `val` be `k`, to get accepted, you need to do the following things:

- Change the array `nums` such that the first `k` elements of `nums` contain the elements which are not equal to `val`. The remaining elements of `nums` are not important as well as the size of `nums`.
- Return `k`.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int val = ...; // Value to remove
int[] expectedNums = [...]; // The expected answer with correct length.
// It is sorted with no values equaling val.

int k = removeElement(nums, val); // Calls your implementation

assert k == expectedNums.length;
sort(nums, 0, k); // Sort the first k elements of nums
for (int i = 0; i < actualLength; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be **accepted**.

Example 1:

Input: `nums = [3,2,2,3], val = 3`
Output: `2, nums = [2,2,_,_]`
Explanation: Your function should return `k = 2`, with the first two elements of `nums` being `2`. It does not matter what you leave beyond the returned `k` (hence they are underscores).

👍 5.4K 🗨️ 995 ☆ 📄 ⓘ

272 Online

</> Code

Java 🔒 Auto

```
1 class Solution {
2     public int removeElement(int[] nums, int val) {
3         int k = 0;
4
5         for (int i = 0; i < nums.length; i++) {
6             if (nums[i] != val) {
7                 nums[k] = nums[i];
8                 k++;
9             }
10        }
11
12        return k;
13    }
14 }
15
```

Saved

Ln 15, Col 1

✅ Testcase ➤ Test Result

Accepted Runtime: 0 ms

✅ Case 1 ✅ Case 2

Input

nums =
[3,2,2,3]

val =
3

Output

[2,2]

Expected

[2,2]

← → ↺ 🌐 leetcode.com/problems/remove-duplicates-from-sorted-array/ ☆ ⬇️ Ⓜ️ ⋮

🏠 Problem List < > ⚙️

🔍 Search

👤 Submit 📄 ⚙️

📊 0 🔔 👤 🌟 Premium

Description Editorial Solutions Submissions

< > </> Code

26. Remove Duplicates from Sorted Array

Easy Topics Companies Hint

Given an integer array `nums` sorted in **non-decreasing order**, remove the duplicates **in-place** such that each unique element appears only **once**. The **relative order** of the elements should be kept the **same**.

Consider the number of *unique elements* in `nums` to be `k`. After removing duplicates, return the number of unique elements `k`.

The first `k` elements of `nums` should contain the unique numbers in **sorted order**. The remaining elements beyond index `k - 1` can be ignored.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length

int k = removeDuplicates(nums); // Calls your implementation

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be **accepted**.

Example 1:

Input: `nums = [1,1,2]`
Output: `2`, `nums = [1,2,_]`
Explanation: Your function should return `k = 2`, with the first two elements of `nums` being 1 and 2 respectively.
It does not matter what you leave beyond the returned `k` (hence they are underscores).

Example 2:

Input: `nums = [0,0,1,1,1,2,2,3,3,4]`

Java Auto

```
1 class Solution {
2     public int removeDuplicates(int[] nums) {
3         int k = 1;
4
5         for (int i = 1; i < nums.length; i++) {
6             if (nums[i] != nums[i - 1]) {
7                 nums[k] = nums[i];
8                 k++;
9             }
10        }
11
12        return k;
13    }
14 }
15
```

Saved Ln 15, Col 1

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums =
[1,1,2]

Output

[1,2]

Expected

[1,2]

👤 Contribute a testcase

👍 19.6K 🗨️ 1.1K ☆ 📄 Ⓜ️ 456 Online

https://leetcode.com/problems/remove-duplicates-from-sorted-array/